

PROGRAM C: WORST FIT

C PROGRAM :

```
#include <stdio.h>
```

```
int main() {
```

```
    int blockSize[20], processSize[20];
```

```
    int allocation[20];
```

```
    int nb, np, i, j, worstIdx;
```

```
    printf("Enter Number of Blocks: ");
```

```
    scanf("%d", &nb);
```

```
    printf("Enter Number of Processes: ");
```

```
    scanf("%d", &np);
```

```
    printf("Enter Block Sizes:\n");
```

```
    for (i = 0; i < nb; i++) {
```

```
        scanf("%d", &blockSize[i]);
```

```
    }
```

```
    printf("Enter Process Sizes:\n");
```

```
    for (i = 0; i < np; i++) {
```

```
        scanf("%d", &processSize[i]);
```

```
    }
```

```
    for (i = 0; i < np; i++) {
```

```
        allocation[i] = -1;
```

```
    }
```

```
    for (i = 0; i < np; i++) {
```

```
        worstIdx = -1;
```

```
        for (j = 0; j < nb; j++) {
```

```
            if (blockSize[j] >= processSize[i]) {
```

```
                if (worstIdx == -1 || blockSize[j] > blockSize[worstIdx]) {
```

```
                    worstIdx = j;
```

```
                }
```

```

    }
}
if (worstIdx != -1) {
    allocation[i] = worstIdx;
    blockSize[worstIdx] -= processSize[i];
}
}
printf("\nProcess No\tProcess Size\tBlock No\n");
for (i = 0; i < np; i++) {
    printf("%d\t\t%d\t\t", i + 1, processSize[i]);
    if (allocation[i] != -1) {
        printf("%d\n", allocation[i] + 1);
    }
    else {
        printf("Not Allocated\n");
    }
}
return 0;

```

}

SHELL SCRIPT :

```
GNU nano 8.7.1
#!/bin/bash

echo "Worst Fit Memory Allocation Demonstration"

blocks=(100 500 200 300 600)
processes=(212 417 112 426)

echo "Memory Blocks: ${blocks[@]}"
echo "Processes: ${processes[@]}"

echo "Allocation Performed Using Worst Fit"
```

Output :

```
(ganesh@LAPTOP-R3RDH9NG)~$ nano worst.sh

(ganesh@LAPTOP-R3RDH9NG)~$ chmod +x worst.sh

(ganesh@LAPTOP-R3RDH9NG)~$ bash worst.sh
Worst Fit Memory Allocation Demonstration
Memory Blocks: 100 500 200 300 600
Processes: 212 417 112 426
Allocation Performed Using Worst Fit
```